

Capgemini Most Asked Java DSA Questions

(Arrays, Strings, Stack, Queue, Trees, BST, Graphs, DP, Bit Manipulation)

Abhishek Rathor

Instagram id: SYNTAX ERROR (code.abhii07)

Key Features:

- **75+ Interview Questions** from Capgemini coding rounds.
- **Beginner to Advanced Topics:** Arrays → Strings → Stack → Queue → Trees → BST → Graphs → DP → Bit Manipulation.
- **Code in Java** with **Time & Space Complexity**.
- **Concept Explanations** included for quick revision.
- **Optimized Solutions** for interviews and campus placements.

Why This Guide?

- **Save Time:** No need to search multiple sources.
- **Practice Smart:** Focus on **most frequently asked questions**.
- **Learn Concepts:** Step-by-step explanations for each topic.
- **Placement Ready:** Covers all Capgemini rounds and coding challenges.

Success is not just about writing code, it's about solving problems.

ARRAYS (10 Questions)

Q1. Two Sum - Find pair with given sum

You are given an array of integers and a target sum. Find two numbers that add up to the target.

Solution:

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{};
}
```

Explanation: Uses hash map to store visited elements. For each element, checks if its complement exists in the map. Time complexity is $O(n)$ with $O(n)$ space.

Q2. Maximum Subarray Sum (Kadane's Algorithm)

You are given an array of integers. Find the contiguous subarray with the maximum sum.

Solution:

```
public int maxSubArray(int[] nums) {
    int maxSum = nums[0], currentSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]);
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
```

Explanation: Maintains current sum and global maximum sum. At each position, decides whether to extend current subarray or start new one. Time complexity $O(n)$.

Q3. Rotate Array

You are given an array and an integer k. Rotate the array to the right by k steps.

Solution:

```
public void rotate(int[] nums, int k) {
    k = k % nums.length;
    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}

private void reverse(int[] nums, int start, int end) {
    while (start < end) {
        int temp = nums[start];
        nums[start] = nums[end];
        nums[end] = temp;
        start++;
        end--;
    }
}
```

Explanation: Uses three reversals to achieve rotation. First reverses entire array, then first k elements, then remaining elements. Time $O(n)$, space $O(1)$.

Q4. Find Duplicate Number

You are given an array containing $n+1$ integers where each integer is between 1 and n . Find the duplicate number.

Solution:

```

public int findDuplicate(int[] nums) {
    int slow = nums[0], fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}

```

Explanation: Uses Floyd's cycle detection algorithm treating array as linked list. Fast and slow pointers detect cycle, then finds cycle start which is the duplicate. $O(n)$ time, $O(1)$ space.

Q5. Merge Sorted Arrays

You are given two sorted arrays. Merge them into a single sorted array.

Solution:

```

public void merge(int[] nums1, int m, int[] nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    while (i >= 0 && j >= 0) {
        if (nums1[i] > nums2[j])
            nums1[k--] = nums1[i--];
        else
            nums1[k--] = nums2[j--];
    }
    while (j >= 0)
        nums1[k--] = nums2[j--];
}

```

Explanation: Merges from end to beginning to avoid overwriting elements. Compares elements from both arrays and places larger one at the end. Time $O(m+n)$, space $O(1)$.

Q6. Best Time to Buy and Sell Stock

You are given an array of stock prices. Find the maximum profit from one buy and one sell transaction.

Solution:

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE, maxProfit = 0;
    for (int price : prices) {
        minPrice = Math.min(minPrice, price);
        maxProfit = Math.max(maxProfit, price - minPrice);
    }
    return maxProfit;
}
```

Explanation: Tracks minimum price seen so far and calculates profit at each point. Updates maximum profit if current profit is higher. Time $O(n)$, space $O(1)$.

Q7. Move Zeros to End

You are given an array with zeros and non-zeros. Move all zeros to the end maintaining relative order.

Solution:

```
public void moveZeroes(int[] nums) {
    int j = 0;
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] != 0) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
            j++;
        }
    }
}
```

Explanation: Uses two pointers where j tracks position for next non-zero element. Swaps non-zero elements to front maintaining order. Time $O(n)$, space $O(1)$.

Q8. Product of Array Except Self

You are given an array. Return array where each element is the product of all other elements.

Solution:

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, 1);

    int left = 1, right = 1;
    for (int i = 0; i < n; i++) {
        result[i] *= left;
        left *= nums[i];
    }
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= right;
        right *= nums[i];
    }
    return result;
}
```

Explanation: First pass calculates left products, second pass calculates right products and multiplies. Avoids division and handles zeros correctly. Time $O(n)$, space $O(1)$ excluding output.

Q9. Find Missing Number

You are given an array containing n distinct numbers from 0 to n. Find the missing number.

Solution:

```
public int missingNumber(int[] nums) {
    int n = nums.length;
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for (int num : nums)
        actualSum += num;
    return expectedSum - actualSum;
}
```

Explanation: Calculates expected sum using formula $n(n+1)/2$ and subtracts actual sum. The difference is the missing number. Time $O(n)$, space $O(1)$.

Q10. Maximum Product Subarray

You are given an array of integers. Find the contiguous subarray with the maximum product.

Solution:

```
public int maxProduct(int[] nums) {
    int maxProd = nums[0], minProd = nums[0], result = nums[0];
    for (int i = 1; i < nums.length; i++) {
        if (nums[i] < 0) {
            int temp = maxProd;
            maxProd = minProd;
            minProd = temp;
        }
        maxProd = Math.max(nums[i], maxProd * nums[i]);
        minProd = Math.min(nums[i], minProd * nums[i]);
        result = Math.max(result, maxProd);
    }
    return result;
}
```

Explanation: Tracks both maximum and minimum products as negatives can become positives. Swaps when encountering negative number. Time $O(n)$, space $O(1)$.

STRINGS (10 Questions)

Q11. Valid Palindrome

You are given a string. Check if it is a palindrome considering only alphanumeric characters and ignoring case.

Solution:

Q11. Valid Palindrome

```
public boolean isPalindrome(String s) {  
    int left = 0, right = s.length() - 1;  
    while (left < right) {  
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;  
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;  
        if (Character.toLowerCase(s.charAt(left)) != Character.toLowerCase(s.charAt(right)))  
            return false;  
        left++;  
        right--;  
    }  
    return true;  
}
```

Explanation: Uses two pointers from both ends, skips non-alphanumeric characters. Compares characters after converting to lowercase. Time $O(n)$, space $O(1)$.

Q12. Longest Substring Without Repeating Characters

You are given a string. Find the length of the longest substring without repeating characters.

Solution:

```
// Q12: Longest Substring Without Repeating Characters  
public int lengthOfLongestSubstring(String s) {  
    Map<Character, Integer> map = new HashMap<>();  
    int maxLen = 0, start = 0;  
    for (int end = 0; end < s.length(); end++) {  
        if (map.containsKey(s.charAt(end)))  
            start = Math.max(start, map.get(s.charAt(end)) + 1);  
        map.put(s.charAt(end), end);  
        maxLen = Math.max(maxLen, end - start + 1);  
    }  
    return maxLen;  
}
```

Explanation: Uses sliding window with hash map to track last seen positions. Moves start pointer when duplicate found. Time $O(n)$, space $O(\min(m,n))$.

Q13. Valid Anagram

You are given two strings. Check if they are anagrams of each other.

Solution:

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    Map<Character, Integer> count = new HashMap<>();
    for (char c : s.toCharArray())
        count.put(c, count.getOrDefault(c, 0) + 1);
    for (char c : t.toCharArray()) {
        count.put(c, count.getOrDefault(c, 0) - 1);
        if (count.get(c) < 0) return false;
    }
    return true;
}
```

Explanation: Counts frequency of characters in first string and decrements for second string. If any count goes negative, not an anagram. Time $O(n)$, space $O(1)$.

Q14. Group Anagrams

You are given an array of strings. Group anagrams together.

Solution:

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] chars = s.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Explanation: Uses sorted string as key to group anagrams in hash map. All anagrams have same sorted form. Time $O(n \cdot k \log k)$ where k is max string length.

Q15. Longest Palindromic Substring

You are given a string. Find the longest palindromic substring.

Solution:

```
public String longestPalindrome(String s) {
    int start = 0, maxLen = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expandAroundCenter(s, i, i);
        int len2 = expandAroundCenter(s, i, i + 1);
        int len = Math.max(len1, len2);
        if (len > maxLen) {
            maxLen = len;
            start = i - (len - 1) / 2;
        }
    }
    return s.substring(start, start + maxLen);
}
```

```
private int expandAroundCenter(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--;
        right++;
    }
    return right - left - 1;
}
```

Explanation: Expands around each center (odd and even length palindromes). Tracks longest palindrome found. Time $O(n^2)$, space $O(1)$.

Q16. String to Integer (atoi)

You are given a string. Convert it to a 32-bit signed integer following atoi rules.

Solution:

```

public int myAtoi(String s) {
    int i = 0, sign = 1;
    long result = 0;

    while (i < s.length() && s.charAt(i) == ' ') i++;
    if (i < s.length() && (s.charAt(i) == '+' || s.charAt(i) == '-'))
        sign = s.charAt(i++) == '-' ? -1 : 1;

    while (i < s.length() && Character.isDigit(s.charAt(i))) {
        result = result * 10 + (s.charAt(i++) - '0');
        if (result * sign > Integer.MAX_VALUE) return Integer.MAX_VALUE;
        if (result * sign < Integer.MIN_VALUE) return Integer.MIN_VALUE;
    }
    return (int) (result * sign);
}

```

Explanation: Handles whitespace, sign, and digits while checking overflow. Returns clamped value if overflow occurs. Time $O(n)$, space $O(1)$.

Q17. Count and Say

You are given an integer n . Generate the n th term of the count-and-say sequence.

Solution:

```

public String countAndSay(int n) {
    String result = "1";
    for (int i = 1; i < n; i++) {
        StringBuilder next = new StringBuilder();
        int count = 1;
        for (int j = 1; j < result.length(); j++) {
            if (result.charAt(j) == result.charAt(j - 1)) {
                count++;
            } else {
                next.append(count).append(result.charAt(j - 1));
                count = 1;
            }
        }
        next.append(count).append(result.charAt(result.length() - 1));
        result = next.toString();
    }
    return result;
}

```

Explanation: Iteratively builds sequence by counting consecutive digits. Appends count followed by digit to next term. Time $O(n*m)$ where m is length.

Q18. Reverse Words in String

You are given a string containing words separated by spaces. Reverse the order of words.

Solution:

```
public String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    StringBuilder result = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        result.append(words[i]);
        if (i > 0) result.append(" ");
    }
    return result.toString();
}
```

Explanation: Reverses entire string, then reverses each word individually. Removes extra spaces in-place. Time $O(n)$, space $O(1)$.

Q19. Implement strStr (Find Needle in Haystack)

You are given two strings haystack and needle. Find the first occurrence of needle in haystack.

Solution:

```
public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;
    int m = haystack.length(), n = needle.length();
    for (int i = 0; i <= m - n; i++) {
        int j = 0;
        while (j < n && haystack.charAt(i + j) == needle.charAt(j))
            j++;
        if (j == n) return i;
    }
    return -1;
}
```

Explanation: Checks each position in haystack for match with needle. Returns index of first match or -1. Time $O(m*n)$, can optimize with KMP.

Q20. Longest Common Prefix

You are given an array of strings. Find the longest common prefix string.

Solution:

```
public String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        int j = 0;
        while (j < prefix.length() && j < strs[i].length() &&
            prefix.charAt(j) == strs[i].charAt(j))
            j++;
        prefix = prefix.substring(0, j);
        if (prefix.isEmpty()) break;
    }
    return prefix;
}
```

Explanation: Takes first string as prefix and compares with each string. Shrinks prefix when mismatch found. Time $O(S)$ where S is sum of all characters.

STACKS (8 Questions)

Q21. Valid Parentheses

You are given a string containing brackets. Check if brackets are properly closed and nested.

Solution:

```

public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    Map<Character, Character> pairs = new HashMap<>();
    pairs.put(')', '(');
    pairs.put(']', '[');
    pairs.put('}', '{');

    for (char c : s.toCharArray()) {
        if (pairs.containsKey(c)) {
            if (stack.isEmpty() || stack.pop() != pairs.get(c))
                return false;
        } else {
            stack.push(c);
        }
    }
    return stack.isEmpty();
}

```

Explanation: Uses stack to match opening and closing brackets. Pushes opening brackets and pops when matching closing found. Time $O(n)$, space $O(n)$.

Q22. Min Stack

You are required to design a stack that supports push, pop, top, and retrieving minimum element in constant time.

Solution:

```

class MinStack {
    Stack<Integer> stack;
    Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val <= minStack.peek())
            minStack.push(val);
    }

    public void pop() {
        if (stack.pop().equals(minStack.peek()))
            minStack.pop();
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}

```

Explanation: Uses two stacks, one for elements and one for tracking minimums. Min stack only stores values when new minimum is found. All operations $O(1)$.

Q23. Evaluate Reverse Polish Notation

You are given an array of strings representing arithmetic expression in Reverse Polish Notation. Evaluate it.

Solution:

```
// Q23. Evaluate Reverse Polish Notation
public int evalRPN(String[] tokens) {
    Stack<Integer> stack = new Stack<>();
    for (String token : tokens) {
        if (token.equals("+") || token.equals("-") ||
            token.equals("*") || token.equals("/")) {
            int b = stack.pop();
            int a = stack.pop();
            if (token.equals("+")) stack.push(a + b);
            else if (token.equals("-")) stack.push(a - b);
            else if (token.equals("*")) stack.push(a * b);
            else stack.push(a / b);
        } else {
            stack.push(Integer.parseInt(token));
        }
    }
    return stack.pop();
}
```

Explanation: Uses stack to store operands. When operator encountered, pops two operands and pushes result. Final stack top is answer. Time $O(n)$.

Q24. Daily Temperatures

You are given an array of temperatures. Return array where each element is number of days until warmer temperature.

Solution:

```

public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] result = new int[n];
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
            int idx = stack.pop();
            result[idx] = i - idx;
        }
        stack.push(i);
    }
    return result;
}

```

Explanation: Uses monotonic stack storing indices. When higher temperature found, pops and calculates days difference. Time $O(n)$, space $O(n)$.

Q25. Next Greater Element

You are given an array. For each element, find the next greater element to its right.

Solution:

```

public int[] nextGreaterElements(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Stack<Integer> stack = new Stack<>();

    for (int i = 2 * n - 1; i >= 0; i--) {
        while (!stack.isEmpty() && stack.peek() <= nums[i % n])
            stack.pop();
        if (i < n && !stack.isEmpty())
            result[i] = stack.peek();
        stack.push(nums[i % n]);
    }
    return result;
}

```

Explanation: Traverses array twice (circular) from right using stack. Maintains decreasing monotonic stack. Time $O(n)$, space $O(n)$.

Q26. Largest Rectangle in Histogram

You are given heights of bars in histogram. Find the largest rectangle area.

Solution:

```
public int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int maxArea = 0, n = heights.length;

    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while (!stack.isEmpty() && h < heights[stack.peek()]) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

Explanation: Uses stack to track indices of increasing heights. Calculates area when smaller height encountered. Time $O(n)$, space $O(n)$.

Q27. Simplify Path

You are given an absolute Unix-style file path. Simplify it to canonical path.

Solution:

```

public String simplifyPath(String path) {
    Stack<String> stack = new Stack<>();
    String[] tokens = path.split("/");

    for (String token : tokens) {
        if (token.equals("") || token.equals(".")) continue;
        if (token.equals("..")) {
            if (!stack.isEmpty()) stack.pop();
        } else {
            stack.push(token);
        }
    }

    StringBuilder result = new StringBuilder();
    for (String dir : stack) {
        result.append("/").append(dir);
    }
    return result.length() > 0 ? result.toString() : "/";
}

```

Explanation: Splits path by '/' and uses stack to handle directory navigation. Handles '..' by popping and ignores '.'. Time O(n), space O(n).

Q28. Decode String

You are given an encoded string with pattern k[encoded_string]. Decode and return the string.

Solution:

```

public String decodeString(String s) {
    Stack<Integer> countStack = new Stack<>();
    Stack<String> strStack = new Stack<>();
    String current = "";
    int k = 0;

    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            k = k * 10 + (c - '0');
        } else if (c == '[') {
            countStack.push(k);
            strStack.push(current);
            current = "";
            k = 0;
        } else if (c == ']') {
            String temp = current;
            current = strStack.pop();
            int count = countStack.pop();
            for (int i = 0; i < count; i++)
                current += temp;
        } else {
            current += c;
        }
    }
    return current;
}

```

Explanation: Uses two stacks for counts and strings. Builds string and expands when closing bracket encountered. Time $O(n)$, space $O(n)$.

QUEUES (5 Questions)

Q29. Implement Queue using Stacks

You are required to implement a queue using only two stacks.

Solution:

```
class MyQueue {
    Stack<Integer> s1, s2;

    public MyQueue() {
        s1 = new Stack<>();
        s2 = new Stack<>();
    }

    public void push(int x) {
        s1.push(x);
    }

    public int pop() {
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) {
                s2.push(s1.pop());
            }
        }
        return s2.pop();
    }

    public int peek() {
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) {
                s2.push(s1.pop());
            }
        }
        return s2.peek();
    }

    public boolean empty() {
        return s1.isEmpty() && s2.isEmpty();
    }
}
```

Explanation: Uses input stack for enqueue and output stack for dequeue. Transfers elements when output stack empty. Amortized $O(1)$ for all operations.

Q30. Sliding Window Maximum

You are given an array and window size k . Find maximum in each sliding window.

Solution:

```
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> deque = new LinkedList<>();
    List<Integer> result = new ArrayList<>();

    for (int i = 0; i < nums.length; i++) {
        while (!deque.isEmpty() && deque.peekFirst() <= i - k)
            deque.pollFirst();
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i])
            deque.pollLast();
        deque.offerLast(i);
        if (i >= k - 1)
            result.add(nums[deque.peekFirst()]);
    }
    return result.stream().mapToInt(i -> i).toArray();
}
```

Explanation: Uses deque to maintain indices in decreasing order of values. Front always contains maximum of current window. Time $O(n)$, space $O(k)$.

Q31. Design Circular Queue

You are required to implement a circular queue with fixed size.

Solution:

```
class MyCircularQueue {
    int[] data;
    int front, rear, size, capacity;

    public MyCircularQueue(int k) {
        data = new int[k];
        front = 0;
        rear = -1;
        size = 0;
        capacity = k;
    }

    public boolean enqueue(int value) {
        if (isFull()) return false;
        rear = (rear + 1) % capacity;
        data[rear] = value;
        size++;
        return true;
    }

    public boolean dequeue() {
        if (isEmpty()) return false;
        front = (front + 1) % capacity;
        size--;
        return true;
    }

    public int Front() {
        return isEmpty() ? -1 : data[front];
    }

    public int Rear() {
        return isEmpty() ? -1 : data[rear];
    }

    public boolean isEmpty() {
        return size == 0;
    }
}
```

Explanation: Uses array with front and rear pointers using modulo for circular behavior. Tracks size separately for full/empty checks. All operations $O(1)$.

Q32. First Non-Repeating Character in Stream

You are given a stream of characters. Find first non-repeating character at each step.

Solution:

```
public String firstNonRepeating(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    Queue<Character> queue = new LinkedList<>();
    StringBuilder result = new StringBuilder();

    for (char c : s.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
        queue.offer(c);
        while (!queue.isEmpty() && freq.get(queue.peek()) > 1)
            queue.poll();
        result.append(queue.isEmpty() ? '#' : queue.peek());
    }
    return result.toString();
}
```

Explanation: Uses queue to maintain order and map for frequency. Removes repeated characters from queue front. Time $O(n)$, space $O(n)$.

Q33. Task Scheduler

You are given tasks and cooldown time n . Find minimum time to complete all tasks.

Solution:

```
public int leastInterval(char[] tasks, int n) {
    Map<Character, Integer> freq = new HashMap<>();
    int maxFreq = 0;
    for (char task : tasks) {
        freq.put(task, freq.getOrDefault(task, 0) + 1);
        maxFreq = Math.max(maxFreq, freq.get(task));
    }
    int maxCount = 0;
    for (int f : freq.values())
        if (f == maxFreq) maxCount++;

    int time = (maxFreq - 1) * (n + 1) + maxCount;
    return Math.max(time, tasks.length);
}
```

Explanation: Calculates minimum intervals based on most frequent task. Formula accounts for cooldown periods between same tasks. Time $O(n)$, space $O(1)$.

RECURSION (8 Questions)

Q34. Fibonacci Number

You are given an integer n . Calculate the n th Fibonacci number using recursion.

Solution:

```
public int fibonacci(int n) {
    Map<Integer, Integer> memo = new HashMap<>();
    return fib(n, memo);
}

private int fib(int n, Map<Integer, Integer> memo) {
    if (n <= 1) return n;
    if (memo.containsKey(n)) return memo.get(n);
    memo.put(n, fib(n - 1, memo) + fib(n - 2, memo));
    return memo.get(n);
}
```

Explanation: Uses memoization to store computed values avoiding recomputation. Base cases are 0 and 1. Time $O(n)$ with memoization, space $O(n)$.

Q35. Power Function

You are given base x and exponent n . Calculate x^n using recursion efficiently.

Solution:

```

public double myPow(double x, int n) {
    if (n == 0) return 1.0;
    if (n < 0) {
        x = 1 / x;
        n = -(n + 1);
        return x * myPow(x, n);
    }
    double half = myPow(x, n / 2);
    if (n % 2 == 0) return half * half;
    return x * half * half;
}

```

Explanation: Uses binary exponentiation dividing problem in half each step. Handles negative exponents by inverting base. Time $O(\log n)$, space $O(\log n)$.

Q36. Generate Parentheses

You are given n pairs of parentheses. Generate all combinations of well-formed parentheses.

Solution:

```

// Q36. Generate Parentheses
public List<String> generateParenthesis(int n) {
    List<String> result = new ArrayList<>();
    generate(n, n, "", result);
    return result;
}

private void generate(int open, int close, String current, List<String> result) {
    if (open == 0 && close == 0) {
        result.add(current);
        return;
    }
    if (open > 0)
        generate(open - 1, close, current + "(", result);
    if (close > open)
        generate(open, close - 1, current + ")", result);
}

```

Explanation: Recursively adds opening brackets when available and closing when valid. Tracks remaining open and close brackets. Time $O(4^n/\sqrt{n})$, space $O(n)$.

Q37. Letter Combinations of Phone Number

You are given a string containing digits 2-9. Return all possible letter combinations from phone keypad.

Solution:

```
public List<String> letterCombinations(String digits) {
    if (digits.isEmpty()) return new ArrayList<>();
    String[] mapping = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "vwxyz"};
    List<String> result = new ArrayList<>();
    backtrack(digits, 0, "", result, mapping);
    return result;
}

private void backtrack(String digits, int idx, String current,
                        List<String> result, String[] mapping) {
    if (idx == digits.length()) {
        result.add(current);
        return;
    }
    String letters = mapping[digits.charAt(idx) - '0'];
    for (char c : letters.toCharArray())
        backtrack(digits, idx + 1, current + c, result, mapping);
}
```

Explanation: Uses backtracking to explore all combinations. Maps digits to letters and recursively builds strings. Time $O(4^n)$, space $O(n)$.

Q38. Permutations

You are given an array of distinct integers. Return all possible permutations.

Solution:

```

public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, result);
    return result;
}

private void backtrack(int[] nums, int start, List<List<Integer>> result) {
    if (start == nums.length) {
        List<Integer> list = new ArrayList<>();
        for (int num : nums) list.add(num);
        result.add(list);
        return;
    }
    for (int i = start; i < nums.length; i++) {
        swap(nums, start, i);
        backtrack(nums, start + 1, result);
        swap(nums, start, i);
    }
}

private void swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}

```

Explanation: Swaps elements to generate permutations recursively. Backtracks by swapping back to original state. Time $O(n! \cdot n)$, space $O(n)$.

Q39. Subsets

You are given an array of unique integers. Return all possible subsets (power set).

Solution:

```

public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int start, List<Integer> current,
    List<List<Integer>> result) {
    result.add(new ArrayList<>(current));
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        backtrack(nums, i + 1, current, result);
        current.remove(current.size() - 1);
    }
}

```

Explanation: Generates subsets by including or excluding each element. Explores all combinations recursively. Time $O(2^n)$, space $O(n)$.

Q40. Combination Sum

You are given array of candidates and target. Find all unique combinations that sum to target.

Solution:

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(candidates, target, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] candidates, int target, int start,
                        List<Integer> current, List<List<Integer>> result) {
    if (target == 0) {
        result.add(new ArrayList<>(current));
        return;
    }
    if (target < 0) return;
    for (int i = start; i < candidates.length; i++) {
        current.add(candidates[i]);
        backtrack(candidates, target - candidates[i], i, current, result);
        current.remove(current.size() - 1);
    }
}
```

Explanation: Uses backtracking allowing same element multiple times. Prunes when target becomes negative. Time $O(n^{(t/m)})$, space $O(t/m)$.

Q41. Word Search

You are given a 2D board and a word. Check if word exists in the grid.

Solution:

```
public boolean exist(char[][] board, String word) {
    for (int i = 0; i < board.length; i++)
        for (int j = 0; j < board[0].length; j++)
            if (dfs(board, word, i, j, 0)) return true;
    return false;
}

private boolean dfs(char[][] board, String word, int i, int j, int idx) {
    if (idx == word.length()) return true;
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length ||
        board[i][j] != word.charAt(idx)) return false;

    char temp = board[i][j];
    board[i][j] = '#';
    boolean found = dfs(board, word, i + 1, j, idx + 1) ||
                    dfs(board, word, i - 1, j, idx + 1) ||
                    dfs(board, word, i, j + 1, idx + 1) ||
                    dfs(board, word, i, j - 1, idx + 1);
    board[i][j] = temp;
    return found;
}
```

Explanation: Uses DFS with backtracking to explore all paths. Marks visited cells temporarily to avoid reuse. Time $O(mn4^L)$, space $O(L)$.

BINARY SEARCH TREES (7 Questions)

Q42. Validate Binary Search Tree

You are given root of binary tree. Check if it is a valid BST.

Solution:

```
public boolean isValidBST(TreeNode root) {
    return isValidBST(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

private boolean isValidBST(TreeNode root, long minVal, long maxVal) {
    if (root == null) return true;
    if (root.val <= minVal || root.val >= maxVal) return false;
    return isValidBST(root.left, minVal, root.val) &&
           isValidBST(root.right, root.val, maxVal);
}
```

Explanation: Recursively checks if each node value is within valid range. Left subtree must have values less than node, right greater. Time $O(n)$, space $O(h)$.

Q43. Lowest Common Ancestor of BST

You are given BST and two nodes. Find their lowest common ancestor.

Solution:

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) return null;
    if (root.val > p.val && root.val > q.val)
        return lowestCommonAncestor(root.left, p, q);
    if (root.val < p.val && root.val < q.val)
        return lowestCommonAncestor(root.right, p, q);
    return root;
}
```

Explanation: Uses BST property to navigate. LCA is node where paths to p and q diverge. Time $O(h)$, space $O(h)$.

Q44. Kth Smallest Element in BST

You are given root of BST and integer k. Find the kth smallest element.

Solution:

```
public int kthSmallest(TreeNode root, int k) {
    int[] result = {-1};
    int[] count = {k};
    inorder(root, count, result);
    return result[0];
}
```

```

private void inorder(TreeNode root, int[] k, int[] result) {
    if (root == null) return;
    inorder(root.left, k, result);
    k[0]--;
    if (k[0] == 0) {
        result[0] = root.val;
        return;
    }
    inorder(root.right, k, result);
}

```

Explanation: Performs inorder traversal which visits nodes in sorted order. Decrements k and returns when k becomes 0. Time $O(h+k)$, space $O(h)$.

Q45. Delete Node in BST

You are given root of BST and key. Delete node with given key.

Solution:

```

// Q45. Delete Node in BST
public TreeNode deleteNode(TreeNode root, int key) {
    if (root == null) return null;
    if (key < root.val) {
        root.left = deleteNode(root.left, key);
    } else if (key > root.val) {
        root.right = deleteNode(root.right, key);
    } else {
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;
        TreeNode minNode = root.right;
        while (minNode.left != null) minNode = minNode.left;
        root.val = minNode.val;
        root.right = deleteNode(root.right, minNode.val);
    }
    return root;
}

```

Explanation: Finds node to delete, replaces with inorder successor if has two children. Handles cases of 0, 1, or 2 children. Time $O(h)$, space $O(h)$.

Q46. Convert Sorted Array to BST

You are given sorted array. Convert it to height-balanced BST.

Solution:

```
public TreeNode sortedArrayToBST(int[] nums) {  
    return sortedArrayToBST(nums, 0, nums.length - 1);  
}  
  
private TreeNode sortedArrayToBST(int[] nums, int left, int right) {  
    if (left > right) return null;  
    int mid = left + (right - left) / 2;  
    TreeNode root = new TreeNode(nums[mid]);  
    root.left = sortedArrayToBST(nums, left, mid - 1);  
    root.right = sortedArrayToBST(nums, mid + 1, right);  
    return root;  
}
```

Explanation: Recursively picks middle element as root ensuring balance. Left half becomes left subtree, right half becomes right subtree. Time $O(n)$, space $O(\log n)$.

Q47. Range Sum of BST

You are given BST root and range [low, high]. Find sum of values in range.

Solution:

```
// Q47. Range Sum of BST
public int rangeSumBST(TreeNode root, int low, int high) {
    if (root == null) return 0;
    int sum = 0;
    if (root.val >= low && root.val <= high)
        sum += root.val;
    if (root.val > low)
        sum += rangeSumBST(root.left, low, high);
    if (root.val < high)
        sum += rangeSumBST(root.right, low, high);
    return sum;
}
```

Explanation: Uses BST property to prune search space. Only explores subtrees that can contain values in range. Time $O(n)$, space $O(h)$.

Q48. Insert into BST

You are given root of BST and value to insert. Insert and return modified tree.

Solution:

```
public TreeNode insertIntoBST(TreeNode root, int val) {
    if (root == null) return new TreeNode(val);
    if (val < root.val)
        root.left = insertIntoBST(root.left, val);
    else
        root.right = insertIntoBST(root.right, val);
    return root;
}
```

Explanation: Recursively finds correct position using BST property. Creates new node when nullptr reached. Time $O(h)$, space $O(h)$.

TREES (9 Questions)

Q49. Maximum Depth of Binary Tree

You are given root of binary tree. Find its maximum depth.

```
public int maxDepth(TreeNode root) {  
    if (root == null) return 0;  
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));  
}
```

Explanation: Recursively calculates depth of left and right subtrees. Returns maximum plus one for current node. Time $O(n)$, space $O(h)$.

Q50. Symmetric Tree

You are given root of binary tree. Check if it is symmetric around its center.

Solution:

```
public boolean isSymmetric(TreeNode root) {  
    return root == null || isMirror(root.left, root.right);  
}  
  
private boolean isMirror(TreeNode left, TreeNode right) {  
    if (left == null && right == null) return true;  
    if (left == null || right == null) return false;  
    return left.val == right.val &&  
        isMirror(left.left, right.right) &&  
        isMirror(left.right, right.left);  
}
```

Explanation: Compares left and right subtrees recursively checking mirror property. Left's left must match right's right. Time $O(n)$, space $O(h)$.

Q51. Level Order Traversal

You are given root of binary tree. Return level order traversal of node values.

Solution:

```
public List<List<Integer>> levelOrder(TreeNode root) {
    if (root == null) return new ArrayList<>();
    List<List<Integer>> result = new ArrayList<>();
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Explanation: Uses BFS with queue processing nodes level by level. Tracks level size to group nodes. Time $O(n)$, space $O(n)$.

Q52. Diameter of Binary Tree

You are given root of binary tree. Find length of longest path between any two nodes.

Solution:

```

private int diameter = 0;

public int diameterOfBinaryTree(TreeNode root) {
    diameter = 0;
    height(root);
    return diameter;
}

private int height(TreeNode root) {
    if (root == null) return 0;
    int left = height(root.left);
    int right = height(root.right);
    diameter = Math.max(diameter, left + right);
    return 1 + Math.max(left, right);
}

```

Explanation: Calculates height while tracking maximum path length through each node. Diameter is sum of left and right heights. Time $O(n)$, space $O(h)$.

Q53. Invert Binary Tree

You are given root of binary tree. Invert it (mirror reflection).

Solution:

```

public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = invertTree(root.right);
    root.right = invertTree(temp);
    return root;
}

```

Explanation: Recursively swaps left and right children at each node. Base case returns nullptr for leaf children. Time $O(n)$, space $O(h)$.

Q54. Path Sum

You are given root and target sum. Check if there's root-to-leaf path summing to target.

Solution:

```
public boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) return false;
    if (root.left == null && root.right == null)
        return root.val == targetSum;
    return hasPathSum(root.left, targetSum - root.val) ||
        hasPathSum(root.right, targetSum - root.val);
}
```

Explanation: Recursively subtracts node values from target. Returns true when leaf reached with remaining sum equal to leaf value. Time $O(n)$, space $O(h)$.

Q55. Lowest Common Ancestor of Binary Tree

You are given binary tree and two nodes. Find their lowest common ancestor.

Solution:

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

Explanation: Recursively searches for nodes in left and right subtrees. If found in both, current node is LCA. Time $O(n)$, space $O(h)$.

Q56. Binary Tree Right Side View

You are given root of binary tree. Return values visible from right side.

Solution:

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            if (i == size - 1) result.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
    return result;
}
```

Explanation: Performs level order traversal and picks last node of each level. Uses BFS with queue. Time $O(n)$, space $O(n)$.

Q57. Serialize and Deserialize Binary Tree

You are required to design algorithm to serialize and deserialize binary tree.

Solution:

```

public String serialize(TreeNode root) {
    if (root == null) return "null,";
    return root.val + "," + serialize(root.left) + serialize(root.right);
}

public TreeNode deserialize(String data) {
    int[] idx = {0};
    return deserialize(data, idx);
}

private TreeNode deserialize(String data, int[] idx) {
    if (idx[0] >= data.length()) return null;
    StringBuilder val = new StringBuilder();
    while (data.charAt(idx[0]) != ',')
        val.append(data.charAt(idx[0]++));
    idx[0]++;
    if (val.toString().equals("null")) return null;
    TreeNode root = new TreeNode(Integer.parseInt(val.toString()));
    root.left = deserialize(data, idx);
    root.right = deserialize(data, idx);
    return root;
}

```

Explanation: Uses preorder traversal for serialization with null markers. Deserializes by parsing string recursively. Time $O(n)$, space $O(n)$.

GRAPHS (8 Questions)

Q58. Number of Islands

You are given 2D grid of '1's (land) and '0's (water). Count number of islands.

Solution:

```

public int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(grid, i, j);
            }
        }
    }
    return count;
}

```

```

private void dfs(char[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length ||
        grid[i][j] == '0') return;
    grid[i][j] = '0';
    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
    dfs(grid, i, j + 1);
    dfs(grid, i, j - 1);
}

```

Explanation: Uses DFS to mark all connected land cells. Each DFS call represents one island discovery. Time $O(mn)$, space $O(mn)$.

Q59. Clone Graph

You are given reference of node in connected undirected graph. Return deep copy.

Solution:

```

class Node {
    public int val;
    public List<Node> neighbors;
    public Node(int val) {
        this.val = val;
        neighbors = new ArrayList<>();
    }
}

public Node cloneGraph(Node node) {
    Map<Node, Node> visited = new HashMap<>();
    return cloneGraph(node, visited);
}

private Node cloneGraph(Node node, Map<Node, Node> visited) {
    if (node == null) return null;
    if (visited.containsKey(node)) return visited.get(node);
    Node clone = new Node(node.val);
    visited.put(node, clone);
    for (Node neighbor : node.neighbors)
        clone.neighbors.add(cloneGraph(neighbor, visited));
    return clone;
}

```

Explanation: Uses DFS with hash map to track cloned nodes. Creates copy and recursively clones neighbors. Time $O(n+e)$, space $O(n)$.

Q60. Course Schedule (Cycle Detection)

You are given number of courses and prerequisites. Check if you can finish all courses.

Solution:

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    int[] indegree = new int[numCourses];

    for (int i = 0; i < numCourses; i++)
        graph.add(new ArrayList<>());

    for (int[] pre : prerequisites) {
        graph.get(pre[1]).add(pre[0]);
        indegree[pre[0]]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++)
        if (indegree[i] == 0) queue.offer(i);

    int count = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        count++;
        for (int next : graph.get(course)) {
            if (--indegree[next] == 0)
                queue.offer(next);
        }
    }
    return count == numCourses;
}
```

Explanation: Uses Kahn's algorithm (topological sort) to detect cycles. If all courses processed, no cycle exists. Time $O(v+e)$, space $O(v+e)$.

Q61. Pacific Atlantic Water Flow

You are given $m \times n$ matrix representing heights. Find cells where water can flow to both oceans.

Solution:

```
public List<List<Integer>> pacificAtlantic(int[][] heights) {
    int m = heights.length, n = heights[0].length;
    boolean[][] pacific = new boolean[m][n];
    boolean[][] atlantic = new boolean[m][n];

    for (int i = 0; i < m; i++) {
        dfs(heights, pacific, i, 0, -1);
        dfs(heights, atlantic, i, n - 1, -1);
    }
    for (int j = 0; j < n; j++) {
        dfs(heights, pacific, 0, j, -1);
        dfs(heights, atlantic, m - 1, j, -1);
    }

    List<List<Integer>> result = new ArrayList<>();
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            if (pacific[i][j] && atlantic[i][j])
                result.add(Arrays.asList(i, j));
    return result;
}

private void dfs(int[][] heights, boolean[][] visited, int i, int j, int prev) {
    if (i < 0 || i >= heights.length || j < 0 || j >= heights[0].length ||
        visited[i][j] || heights[i][j] < prev) return;
    visited[i][j] = true;
    dfs(heights, visited, i + 1, j, heights[i][j]);
    dfs(heights, visited, i - 1, j, heights[i][j]);
    dfs(heights, visited, i, j + 1, heights[i][j]);
    dfs(heights, visited, i, j - 1, heights[i][j]);
}
```

Explanation: DFS from ocean edges inward checking reachability. Cells reachable from both oceans are added to result. Time $O(mn)$, space $O(mn)$.

Q62. Shortest Path in Binary Matrix

You are given $n \times n$ binary matrix. Find shortest clear path from top-left to bottom-right.

Solution:

```
public int shortestPathBinaryMatrix(int[][] grid) {
    if (grid[0][0] == 1) return -1;
    int n = grid.length;
    int[][] dirs = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
    Queue<int[]> queue = new LinkedList<>();
    queue.offer(new int[]{0, 0});
    grid[0][0] = 1;
    int steps = 1;

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int[] curr = queue.poll();
            int x = curr[0], y = curr[1];
            if (x == n - 1 && y == n - 1) return steps;
            for (int[] dir : dirs) {
                int nx = x + dir[0], ny = y + dir[1];
                if (nx >= 0 && nx < n && ny >= 0 && ny < n && grid[nx][ny] == 0) {
                    grid[nx][ny] = 1;
                    queue.offer(new int[]{nx, ny});
                }
            }
            steps++;
        }
    }
    return -1;
}
```

Explanation: Uses BFS to find shortest path considering 8 directions. Marks visited cells to avoid revisiting. Time $O(n^2)$, space $O(n^2)$.

Q63. Rotting Oranges

You are given grid with fresh and rotten oranges. Find minimum time for all oranges to rot.

Solution:

```

public int orangesRotting(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    Queue<int[]> queue = new LinkedList<>();
    int fresh = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == 2) queue.offer(new int[]{i, j});
            if (grid[i][j] == 1) fresh++;
        }
    }
    if (fresh == 0) return 0;

    int minutes = 0;
    int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int[] curr = queue.poll();
            int x = curr[0], y = curr[1];
            for (int[] dir : dirs) {
                int nx = x + dir[0], ny = y + dir[1];
                if (nx >= 0 && nx < m && ny >= 0 && ny < n && grid[nx][ny] == 1) {
                    grid[nx][ny] = 2;
                    fresh--;
                    queue.offer(new int[]{nx, ny});
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes - 1 : -1;
}

```

Explanation: Uses multi-source BFS starting from all rotten oranges. Spreads rot level by level tracking time. Time $O(mn)$, space $O(mn)$.

Q64. Word Ladder

You are given beginWord, endWord and wordList. Find shortest transformation sequence length.

Solution:

```
public int ladderLength(String beginWord, String endWord, List<String> wordList) {
    Set<String> dict = new HashSet<>(wordList);
    if (!dict.contains(endWord)) return 0;
    Queue<String> queue = new LinkedList<>();
    queue.offer(beginWord);
    int level = 1;

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            String word = queue.poll();
            if (word.equals(endWord)) return level;
            char[] chars = word.toCharArray();
            for (int j = 0; j < chars.length; j++) {
                char original = chars[j];
                for (char c = 'a'; c <= 'z'; c++) {
                    chars[j] = c;
                    String newWord = new String(chars);
                    if (dict.contains(newWord)) {
                        queue.offer(newWord);
                        dict.remove(newWord);
                    }
                }
                chars[j] = original;
            }
        }
        level++;
    }
    return 0;
}
```

Explanation: Uses BFS trying all one-letter transformations at each level. Erases used words to avoid cycles. Time $O(m^2n)$, space $O(mn)$.

Q65. Graph Valid Tree

You are given n nodes and edges list. Check if edges form valid tree.

Solution:

```
// Q65. Graph Valid Tree  
public boolean validTree(int n, int[][] edges) {  
    if (edges.length != n - 1) return false;  
    int[] parent = new int[n];  
    for (int i = 0; i < n; i++) parent[i] = i;  
  
    for (int[] edge : edges) {  
        int p1 = find(parent, edge[0]);  
        int p2 = find(parent, edge[1]);  
        if (p1 == p2) return false;  
        parent[p1] = p2;  
    }  
    return true;  
}  
  
private int find(int[] parent, int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent, parent[x]);  
    return parent[x];  
}
```

Explanation: Uses union-find to detect cycles. Valid tree has n-1 edges and no cycles. Time $O(n \cdot \alpha(n))$, space $O(n)$.

DYNAMIC PROGRAMMING (5 Questions)

Q66. Climbing Stairs

You are given n steps. Find number of ways to climb using 1 or 2 steps at a time.

Solution:

```
public int climbStairs(int n) {  
    if (n <= 2) return n;  
    int prev2 = 1, prev1 = 2;  
    for (int i = 3; i <= n; i++) {  
        int current = prev1 + prev2;  
        prev2 = prev1;  
        prev1 = current;  
    }  
    return prev1;  
}
```

Explanation: Uses DP similar to Fibonacci. Ways to reach step i is sum of ways to reach $i-1$ and $i-2$. Time $O(n)$, space $O(1)$.

Q67. House Robber

You are given array of house values. Find maximum money you can rob without robbing adjacent houses.

Solution:

```

public int rob(int[] nums) {
    if (nums.length == 0) return 0;
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int current = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}

```

Explanation: At each house, chooses max between robbing current house plus i-2 or skipping it. Uses two variables for optimization. Time $O(n)$, space $O(1)$.

Q68. Coin Change

You are given coins array and amount. Find minimum coins needed to make amount.

Solution:

```

public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin)
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}

```

Explanation: Builds up solutions from 0 to amount. For each amount, tries all coins and picks minimum. Time $O(\text{amount} \times \text{coins})$, space $O(\text{amount})$.

Q69. Longest Increasing Subsequence

You are given array of integers. Find length of longest strictly increasing subsequence.

Solution:

```
public int lengthOfLIS(int[] nums) {
    List<Integer> dp = new ArrayList<>();
    for (int num : nums) {
        int pos = Collections.binarySearch(dp, num);
        if (pos < 0) pos = -(pos + 1);
        if (pos == dp.size())
            dp.add(num);
        else
            dp.set(pos, num);
    }
    return dp.size();
}
```

Explanation: Uses binary search with patience sorting approach. Maintains array of smallest tail elements for each length. Time $O(n \log n)$, space $O(n)$.

Q70. Unique Paths

You are given $m \times n$ grid. Find number of unique paths from top-left to bottom-right.

Solution:

```
public int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j - 1];
        }
    }
    return dp[n - 1];
}
```

Explanation: Uses 1D DP array where each cell is sum of paths from left and top. Space optimized from 2D to 1D. Time $O(m \cdot n)$, space $O(n)$.

BIT MANIPULATION (5 Questions)

Q71. Single Number

You are given array where every element appears twice except one. Find the single element.

Solution:

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) {
        result ^= num;
    }
    return result;
}
```

Explanation: XOR of two same numbers is 0, and XOR of any number with 0 is number itself. All pairs cancel out. Time $O(n)$, space $O(1)$.

Q72. Number of 1 Bits (Hamming Weight)

You are given unsigned integer. Count number of '1' bits.

Solution:

```
public int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
        count++;
        n &= (n - 1);
    }
    return count;
}
```

Explanation: Brian Kernighan's algorithm flips rightmost set bit to 0 each iteration. Counts iterations until n becomes 0. Time $O(\log n)$, space $O(1)$.

Q73. Reverse Bits

You are given 32-bit unsigned integer. Reverse its bits.

Solution:

```
public int reverseBits(int n) {
    int result = 0;
    for (int i = 0; i < 32; i++) {
        result <<= 1;
        result |= (n & 1);
        n >>= 1;
    }
    return result;
}
```

Explanation: Extracts least significant bit of n and adds to result. Shifts result left and n right 32 times. Time O(1), space O(1).

Q74. Power of Two

You are given integer n. Check if it is a power of two.

Solution:

```
public boolean isPowerOfTwo(int n) {  
    return n > 0 && (n & (n - 1)) == 0;  
}
```

Explanation: Power of 2 has only one bit set. n & (n-1) removes rightmost bit, result is 0 for powers of 2. Time O(1), space O(1).

Q75. Missing Number (Bit Manipulation)

You are given array containing n distinct numbers from 0 to n. Find missing number using XOR.

Solution:

```
public int missingNumberXOR(int[] nums) {  
    int result = nums.length;  
    for (int i = 0; i < nums.length; i++) {  
        result ^= i ^ nums[i];  
    }  
    return result;  
}
```

Explanation: XORs all indices and values. Duplicate numbers cancel out leaving only missing number.

Time $O(n)$, space $O(1)$.

 **Congratulations!**

You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills.

Stay connected with us for more **coding tutorials, challenges, tips, and tech content** on Instagram: **@SYNTAXERROR**. Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved.

This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.